

```
#include<stdio.h>
int main(){
int arr[100],n,i,pos,value,choice,found;
printf("enter number of elements : ");
scanf("%d",&n);
printf("enter elements:\n");
for(i=0;i<n;i++){
    scanf("%d",&arr[i]);
}
do{
    printf("\n---MENU---\n");
    printf("1. traversal\n2. Insertion\n3. Deletion\n4. Searching\n5. Exit\n");
    printf("enter your choice: ");
    scanf("%d",&choice);
    switch(choice){
case 1://traversal
    printf("array elements are: ");
    for(i=0;i<n;i++){
        printf("%d",arr[i]);
    }
    break;
case 2://insertion
    printf("enter position and value: ");
    scanf("%d%d",&pos,&value);
    for(i=n;i>=pos;i--){
        arr[i]=arr[i-1];
    }
    arr[pos-1]=value;
```

```
    arr[pos-1]=value;
    n++;
    break;
case 3 : //deletion
    printf("enter the position to delete: ");
    scanf("%d",&pos);
    for(i=pos-1;i<n-1;i++){
        arr[i]=arr[i+1];
    }
    n--;
    break;
case 4: //searching
    printf("enter the element to search: ");
    scanf("%d",&value);
    found=0;
    for(i=0;i<n;i++){
        if(arr[i]==value){
            printf("element found at position %d\\n",i+1);

            found=1;
            break;
        }
    }

    if(found==0)
        printf("element not found\\n");
    break;
```

```
    if(found==0)
        printf("element not found\n");
    break;
case 5 :
    printf("exiting..\n");
    break;
default:
    printf("invalid choice!\n");
}
} while(choice!=5);
return 0;
}
```

```
"C:\Users\Manisha B S\Docum  × + v
enter number of elements : 5
enter elements:
1 2 3 4 5

---MENU---
1. traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
enter your choice: 1
array elements are: 12345
---MENU---
1. traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
enter your choice: 2
enter position and value: 2 3

---MENU---
1. traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
enter your choice: 3
enter the position to delete: 3
```

```
---MENU---
1. traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
enter your choice: 3
enter the position to delete: 3
```

```
---MENU---
1. traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
enter your choice: 4
enter the element to search: 4
element found at position 4
```

n

```
---MENU---
1. traversal
2. Insertion
3. Deletion
4. Searching
5. Exit
enter your choice: 5
exiting..
```

Process returned 0 (0x0) execution time : 60.348 s
Press any key to continue.

```
#include<stdio.h>
int main(){
int matrix[10][10],triplet[50][3],transpose[50][3];
int i,j,m,n,k=1;
printf("enter rows and columns: ");
scanf("%d%d",&m,&n);
printf("enter the matrix elements:\n");
for(i=0;i<m;i++){
    for(j=0;j<n;j++){
        scanf("%d",&matrix[i][j]);
    }
}
//convert to triplet form
triplet[0][0]=m;
triplet[0][1]=n;
triplet[0][2]=0;
for(i=0;i<m;i++){
    for(j=0;j<n;j++){
        if(matrix[i][j]!=0){
            triplet[k][0]=i;
            triplet[k][1]=j;
            triplet[k][2]=matrix[i][j];
            k++;
            triplet[0][2]++;
        }
    }
}
printf("\n triplet representation :\n");
```

```
    }  
}  
printf("\n triplet representation :\n");  
for(i=0;i<k;i++){  
    printf("%d %d %d \n",triplet[i][0],triplet[i][1],triplet[i][2]);  
    //transpose  
    transpose[0][0]=triplet[0][1];  
    transpose[0][1]=triplet[0][0];  
    transpose[0][2]=triplet[0][2];  
    k=1;  
    for(i=0;i<n;i++){  
        for(j=1;j<=triplet[0][2];j++){  
            if(triplet[j][1]==i){  
                transpose[k][0]=triplet[j][1];  
                transpose[k][1]=triplet[j][0];  
                transpose[k][2]=triplet[j][2];  
                k++;  
            }  
        }  
    }  
    printf("\n transpose triplet :\n");  
    for(i=0;i<k;i++){  
        printf("%d %d %d\n",transpose[i][0],transpose[i][1],transpose[i][2]);  
    }  
    return 0;  
}
```

```
enter rows and columns: 2 3
enter the matrix elements:
1 2
3 4
5 6
```

```
triplet representation :
2 3 6
0 0 1
0 1 2
0 2 3
1 0 4
1 1 5
1 2 6
```

```
transpose triplet :
3 2 6
0 0 1
0 1 4
1 0 2
1 1 5
2 0 3
2 1 6
```

```
Process returned 0 (0x0)    execution time : 13.936 s
Press any key to continue.
```



```
#include<stdio.h>
#define MAX 5
int stack[MAX];
int top=-1;
//push
void push(int value){
if(top==MAX-1)
    printf("stack overflow\n");
else{
    top++;
    stack[top]=value;
    printf("pushed %d\n",value);
}
}
//pop
void pop(){
if(top== -1)
    printf("stack underflow\n");
else{
    printf("popped %d\n",stack[top]);
    top--;
}
}
void display(){
if(top== -1)
    printf("stack is empty\n");
else{
    printf("stack element: \n");
}
```

```
void display() {  
    if(top==-1)  
        printf("stack is empty\n");  
    else{  
        printf("stack element: \n");  
        for(int i=top;i>=0;i--)  
            printf("%d\n",stack[i]);  
    }  
}
```

//main

```
int main() {  
    push(10);  
    push(20);  
}
```

pushed 10
pushed 20

Process returned 0 (0x0) execution time : 1.282 s
Press any key to continue.

```
#include<stdio.h>
#include<stdlib.h>
struct Node{
int data;
struct Node*next;
};
struct Node*createNode(int data){
struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
newNode->data=data;
newNode->next=NULL;
return newNode;};
void insertAtBeginning(struct Node**head,int data){
struct Node*newNode=createNode(data);
newNode->next=*head;
*head=newNode;
}
void insertatend(struct Node**head,int data){
struct Node*newNode=createNode(data);
if(*head==NULL){
*head=newNode;
return;}
struct Node*temp=*head;
while(temp->next!=NULL){
temp=temp->next;
}
temp->next=newNode;}
void insertAtposition(struct Node**head,int data,int position){
```

```
ds_github.c x Untitled1.c x Untitled2.c x *Untitled3 x
void insertAtposition(struct Node**head,int data,int position){
struct Node*newNode=createNode(data);
if(position==0){
    newNode->next=*head;
    *head=newNode;
    return;
}
struct Node*temp=*head;
for(int i=0;temp!=NULL&& i<position-1;
i++){
temp=temp->next;
}
if(temp==NULL){
    printf("position out of range\n");
    return ;
}
newNode->next=temp->next;
temp->next=newNode;}
void deleteFromBeginning(struct Node**head){
if(*head==NULL) return;
struct Node*temp=*head;
*head=(*head)->next;
free(temp);}
void deleteFromEnd(struct Node**head){
if(*head==NULL) return;
if ((*head)->next==NULL){
    free(*head);
    *head=NULL;
```

```
github.c x Untitled1.c x Untitled2.c x Untitled3 x
    *head=NULL;
    return;
}
struct Node*temp=*head;
while(temp->next->next!=NULL) {
    temp=temp->next;
}
free(temp->next);
temp->next=NULL;
}
void deleteFromPosition(struct Node**head,int position){
    if(*head==NULL) return;
    struct Node*temp=*head;
    if(position+=0){
        *head=temp->next;
        free(temp);
        return;
    }
    for(int i=0;temp!=NULL&&i<position-1;
    i++){
        temp=temp->next;
    }
    if(temp==NULL||temp->next==NULL){
        printf("position out of range\n");
        return;
    }
    struct Node*next=temp->next->next;
    free(temp->next);
```

```
free(temp->next);
temp->next=next;}
void transverse(struct Node*head){
struct Node*temp=head;
while(temp!=NULL)
{
    printf("%d->",temp->data);
    temp=temp->next;
}
printf("NULL\n");
}
int search(struct Node*head,int target){
struct Node*temp=head;
while(temp!=NULL){
    if(temp->data==target)return 1;
    temp=temp->next;
}
return 0;
}
int main(){
struct Node*head=NULL;
insertatend(&head,10);
insertatend(&head,20);
insertatend(&head,30);
transverse(head);
insertAtBeginning(&head,5);
transverse(head);
insertAtposition(&head,25,2);
```

```
insertAtposition(&head,25,2);
transverse(head);
deleteFromBeginning(&head);
transverse(head);
deleteFromEnd(&head);
transverse(head);
deleteFromPosition(&head,1);
transverse(head);
printf("list contains 20 :%s\n",search(head,20)?"yes" : "No");
printf("list contains 40: %s\n",search(head,40)?"yes" : "No");

return 0;
}
```



```
"C:\Users\Manisha B S\Docum x + v
10->20->30->NULL
5->10->20->30->NULL
5->10->25->20->30->NULL
10->25->20->30->NULL
10->25->20->NULL
25->20->NULL
list contains 20 :yes
list contains 40: No

Process returned 0 (0x0)    execution time : 13.312 s
Press any key to continue.
|
```